

College Micro Economy Simulator: Technical Design Document

Repository link: <https://github.com/kae1506/College-Micro-Economy-Simulator/>

Overview

The **College Policy Simulator** is an AI-driven simulation platform designed to model how institutional policies affect the well-being, performance, and behavior of various stakeholders in a college ecosystem. The simulator allows users (e.g., administrators, researchers, or students) to adjust policy parameters, such as class timings, funding allocation, or extracurricular frequency, and observe their cascading effects across multiple agent types in a simulated environment.

The system's goal is to offer a sandbox for understanding policy outcomes dynamically and ethically through agent-based modeling, reinforcement learning, and LLM reasoning.

System Architecture

1. Frontend

Technology: Streamlit + Plotly :

Streamlit UI: for rapid prototyping, slider-based controls, and real-time interaction. –

Plotly visualizations: to display agent states, environmental parameters, and inter-actor graphs. –

Features:

- Adjustable policy variables.
- Graphs showing changes in actor states over time.
- Node-to-node influence graph.
- Actor-specific dashboard.

2. Backend

Technology: Python + Gemini (LLM wrappers) + PyTorch (RL training)

Gemini-based LLM reasoning layer: interprets new policies and predicts how environmental parameters will change, using LLM querying.

RL module: trains agent behaviors to maximize individual reward functions based on simulated conditions.

Simulation engine: handles time-step progression, updates to actor states, and global environmental changes. This will also function as the main loop.

Data management: in-memory or SQLite database for state tracking.

3. Data Flow

1. User modifies policy variables through Streamlit sliders.
 2. Policies are sent to the LLM reasoning layer to determine changes in global environmental states.
 3. Agents perceive new environment states and take actions using their learned policy (from the RL model).
 4. Actions modify both individual and global states.
 5. Results are visualized in real time.
-

Modules

1. **Frontend (Streamlit)** - UI controls, graphs, dashboards.
 2. **Simulation Engine** - handles major control over all layers.
 3. **Policy Manager** - stores and modifies policy variables.
 4. **Environment Layer** - manages and controls environmental variables and depends on the same
 5. **LLM Reasoning Layer** - uses Gemini API for policy interpretation and world updates.
 6. **Agent Layer** - manages and controls agent-specific variables and agent actions (includes training).
 7. **Visualization Layer** - dynamic charts, network visualization.
 8. **Data Persistence Layer** - saves simulation history.
-

Core Design Components

1. Policy Variables (User Inputs)

Things that can be controlled by the user. For example: Class Timings, Club Fundings, Exam Frequency, Teacher Load per Week, etc.

2. Environment States

Divided into **Global States** and **Actor States**.

Global States: State variables (usually some measurable outcomes) which are generalised over all actors. Such as campus energy, institution reputation, and break frequency.

Actor States: State variables that are specific to actors. For example: performance, happiness, and fatigue for students, teaching quality

3. Action Space

Actions each actor can take: For example: Attend/skip class, Participate in events - Study more / relax more, Request resource or feedback, Organize social event

4. Reward Functions

Each actor type (student, teacher, admin) has a unique reward structure:

Students: maximize happiness + performance – fatigue.

Teachers: maximize student performance + satisfaction – burnout.

Admins: maximize global performance + participation – stress.

Rewards are updated every time step, guiding RL training.

4. Communication Channels

There must be constant, robust, and consistent communication between each layer. The policy layer must directly affect the environment layer, and the environment layer must have a two-way communication channel between itself and the actor layer. For this, a data structure will be defined to make sure that access times are fast, and memory management is prioritised.

Reinforcement Learning Structure

- **Algorithm:** PPO (Proximal Policy Optimization) or DQN (Deep Q-Network).
- **Environment:** Gym-style simulation environment wrapping the college ecosystem.
- **State Representation:** a combination of global and local (actor-specific) variables.
- **Reward:** derived from agent reward functions.

- **Training:** pre-training on synthetic data (policy-outcome datasets), then fine-tuned during simulation.
-

Repository Design

- **data/** – Stores datasets like `variable_history.csv`, tracking key simulation variables over time.
 - **examples/** – Contains demo scripts such as `test_case1.py` showing how to run simulations.
 - **src/** – Core logic, organized into key modules:
 - **actors/** – Defines all participants (students, professors, clubs, administration).
 - **environment/** – Sets up the simulation world and AI interfaces (`llm_wrappers.py`).
 - **policy-layer/** – Manages rules, simulation flow, and learning (`simulation.py`, `training.py`).
 - **utils/** – Provides support tools for communication, visualization, and frontend interaction.
-

Tech Stack Summary

Layer	Technology	Purpose
Frontend	Streamlit, Plotly	Visualization, interactivity
Backend	Python, PyTorch, Gemini	Simulation logic, RL, policy reasoning
Data	SQLite / Pandas	State persistence
Visualization	Plotly, NetworkX	Graphs and network view

Extensions / Future Work

- Add forecasting layer for long-term policy outcomes.
 - Using LLMs to remove hard-coding for state variables and actions.
 - Adaptive reward functions learned from user feedback.
 - Integrate with real anonymized college datasets.
-